

Assignment 1: Calculator Testing

Create class:

Calculator

Methods:

- add
 - subtract
 - multiply
 - divide
-

Tasks:

1. Write **4–5 test cases per method**
 2. Use:
 - assertEquals
 - assertNotEquals
 3. Cover:
 - positive numbers
 - negative numbers
 - zero
-

Thinking:

- What happens when both inputs are negative?
 - What happens when result is zero?
-

Assignment 2: String Utility

Methods:

- isEmpty(String)
 - toUpperCase(String)
 - getLength(String)
-

Tasks:

Use:

- `assertTrue`
- `assertFalse`
- `assertNull`
- `assertNotNull`

Test:

- `null`
 - `empty string`
 - `whitespace`
 - `normal string`
-

Assignment 3: Array Reverse

Method:

```
int[] reverseArray(int[] arr)
```

Tasks:

Use:

- `assertArrayEquals`

Test:

- `normal array`
 - `single element`
 - `empty array`
 - `null (if allowed)`
-

Assignment 4: User Class

Fields:

- name
 - age
-

Tasks:

1. Use:
 - `@BeforeEach`
 - `@AfterEach`
 2. Initialize object in setup
 3. Write tests for:
 - valid user
 - null name
 - invalid age
-

Assignment 5: Bank Account

Methods:

- deposit
 - withdraw
 - getBalance
-

Tasks:

1. Use lifecycle methods
 2. Write tests for:
 - valid operations
 - invalid operations
 3. Validate:
 - balance updates correctly
-

Assignment 6: Grouped Assertions

Task:

Use `assertAll` to validate multiple properties:

Example:

- user name
 - user age
 - user status
-

Assignment 7: Test Naming Practice

Rewrite test names using:

`shouldExpectedBehaviorWhenCondition`

Example:

`shouldReturnUpdatedBalanceWhenDepositIsSuccessful`

Assignment 8: Division Exception

Task:

Write test for:

`divide(a, b)`

Requirements:

- Use `assertThrows`
 - Validate:
 - exception type
 - message
-

Assignment 9: Password Validator

Rules:

- minimum 8 characters
 - at least 1 uppercase
 - at least 1 number
-

Tasks:

Write tests for:

- valid password
 - each invalid rule
 - null
 - empty
-

Assignment 10: Boundary Testing

Method:

```
boolean isValidMarks(int marks)
```

Valid range: 0-100

Tasks:

Test:

- 0
 - 1
 - 99
 - 100
 - -1
 - 101
-

Assignment 11: Multiple Exception Scenarios

Create method:

```
withdraw(double balance, double amount)
```

Rules:

- $\text{amount} > \text{balance} \rightarrow \text{exception}$
 - $\text{amount} \leq 0 \rightarrow \text{exception}$
-

Tasks:

Write separate test methods for each exception scenario.